

Exemplo de derivação sintática: for em Pascal

Objetivo: mostrar como um pequeno trecho de código pode ser construído a partir de regras gramaticais de uma linguagem real.

Fonte de referência: ECMA-262 — ECMAScript® Language Specification, seções "Declarations and the Variable Statement" (Lexical Declarations) e "Arrow Function Definitions". Disponível em <https://tc39.es/ecma262/>.

A notação utilizada pela especificação é uma gramática livre de contexto no estilo BNF (Backus-Naur Form), com produções léxicas e sintáticas. A gramática abaixo foi simplificada para fins didáticos, preservando a estrutura sintática essencial da declaração escolhida.

1. Código gerado

```
const dobro = (x) => x * 2;
```

2. Regras gramaticais utilizadas

Para concentrar a derivação na declaração da constante, usamos uma gramática reduzida. Os símbolos entre < > são não terminais; os demais representam terminais da linguagem (const, =, =>, *, ,, parênteses, o identificador x, a constante dobro e o literal 2).

```
<lexical_declaration> ::= <let_or_const> <binding_list>;  
<let_or_const> ::= const  
<binding_list> ::= <lexical_binding>  
<lexical_binding> ::= <binding_identifier> = <initializer>  
<initializer> ::= <arrow_function>  
<arrow_function> ::= <arrow_parameters> => <concise_body>  
<arrow_parameters> ::= ( <formal_parameter_list> )  
<formal_parameter_list> ::= <binding_identifier>  
<concise_body> ::= <expression>  
<expression> ::= <identifier_reference> | <numeric_literal> | <expression>  
                  | <mult_operator> <expression>  
<mult_operator> ::= *  
<binding_identifier> ::= dobro | x  
<identifier_reference> ::= x  
<numeric_literal> ::= 2
```

3. Derivação

A derivação abaixo começa no símbolo não terminal <lexical_declaration> e substitui, passo a passo, cada não terminal até obter a sequência de terminais correspondente ao trecho desejado.

```

<lexical_declaration>
⇒ <let_or_const> <binding_list> ;
⇒ const <binding_list> ;
⇒ const <lexical_binding> ;
⇒ const <binding_identifier> = <initializer> ;
⇒ const dobro = <initializer> ;
⇒ const dobro = <arrow_function> ;
⇒ const dobro = <arrow_parameters> => <concise_body> ;
⇒ const dobro = ( <formal_parameter_list> ) => <concise_body> ;
⇒ const dobro = ( <binding_identifier> ) => <concise_body> ;
⇒ const dobro = ( x ) => <concise_body> ;
⇒ const dobro = ( x ) => <expression> ;
⇒ const dobro = ( x ) => <expression> <mult_operator> <expression> ;
⇒ const dobro = ( x ) => <identifier_reference> <mult_operator> <expression> ;
⇒ const dobro = ( x ) => x <mult_operator> <expression> ;
⇒ const dobro = ( x ) => x * <expression> ;
⇒ const dobro = ( x ) => x * <numeric_literal> ;
⇒ const dobro = ( x ) => x * 2 ;

```

Forma concreta em JavaScript, com pontuação:

```
const dobro = (x) => x * 2;
```

4. Breve Explicação Textual

A declaração `const` de JavaScript cria uma ligação (binding) imutável, usada aqui para armazenar uma função. Nesse exemplo, `dobro` recebe uma arrow function definida entre parênteses, `(x)`, que lista seu único parâmetro. Após o símbolo `=>`, aparece o corpo conciso (concise body) da função: como não há chaves `{ }`, a expressão à direita — `x * 2` — é automaticamente o valor retornado, sem a necessidade da palavra-chave `return`. A derivação demonstra que o código não surge arbitrariamente: ele é uma sentença válida porque pode ser obtida por substituições sucessivas a partir das regras da gramática, partindo de `<lexical_declaration>` e chegando à sequência de terminais `const dobro = (x) => x * 2 ;`.

Referência: ECMA International. ECMA-262, ECMAScript® Language Specification — seções 14.3.1 (Let and Const Declarations) e 15.3 (Arrow Function Definitions). Disponível em: <https://tc39.es/ecma262/>.